



CRATE IQ: Firmware & System Architecture Specification

This document defines the software stack for the CRATE IQ pocket device, ensuring high-speed on-device intelligence with minimal power consumption

1. Software Stack Overview

Layer	Component	Description
Application	IntelligenceEngine	Core logic for OCR matching and metadata retrieval.
Middleware	UI Framework(1-bit)	Custom monochrome layout engine for Sharp Memory LCD.
Intelligence	Embedded OCR (Tesseractlite)	Local ML model for text extraction from images
Database	Indexed Music DB (SQLite)	Local lookup table for credits, personnel and lineage.
OS Layer	RTOS / Embedded Linux	Managing power states, filesystem and USB-C stack.
HAL / Drivers	Camera, Display, Haptics	Direct hardware control and peripheral drivers

2. Intelligence Engine Flow (Aim-Scan-Reveal)

1. Capture Phase:
- Triggered by physical button interrupt
 - Camera wakes up and performs fast autofocus (AF).
 - Image captured into frame buffer (13MP RAW -> 2k/4k Processing).
2. OCR Processing:
- Raw image is converted to grayscale and thresholded
 - Localized OCR identifies key text fields (Album Title, Artist, Catalog #).
3. Metadata Lookup:
- Fuzzy matching against the local SQLite database
 - Retrieves "Credits-First" payload: Personnel, Studio, Sample Info.
4. UI Rendering:
- Payload is formatted using the 'Crate-Sans' bitmap font system
 - Full 400x240 bit-map buffer is written to Sharp Memory LCD via SPI.
5. Sleep Phase:
- Peripherals enter deep sleep immediately after UI write.

3. Database Schema Concept

The local database must be optimized for fast indexed lookups.

```
CREATE TABLE records (  
    id INTEGER PRIMARY KEY,  
    title TEXT,  
    artist TEXT,  
    label TEXT,  
    release_year INTEGER,  
    studio TEXT  
);  
  
-- Personnel (Many-to-Many)  
CREATE TABLE personnel (  
    id INTEGER PRIMARY KEY,  
    name TEXT,  
    role TEXT,  
    record_id INTEGER,  
    FOREIGN KEY(record_id) REFERENCES records(id)  
);  
  
-- Sample Lineage  
CREATE TABLE lineage (  
    id INTEGER PRIMARY KEY,  
    source_record_id INTEGER,  
    target_record_id INTEGER,  
    description TEXT  
);
```

4. Power State Management

State	Power Draw	Wake-Up Time	Trigger
Active	~150mW	N/A	Button Pressed
Static View	~50uW	<10ms	After UI Write
Deep Sleep	~5uW	<200ms	30s Idle

5. Next Execution Steps

- ☐ **Prototype Intelligence Engine:** Develop a Python simulator for the metadata matching logic.
- ☐ **Data Pipeline:** Design the "Library Sync" tool for populating the local database via USB-C.
- ☐ **UI Rendering Mockup:** Code a bit-map generator for the 400x240 monochrome display.